RAPPORT TECHNIQUE

ADRIA BANKING AI

Architecture microservices et assistant bancaire

Spring Boot - Spring Cloud - REST - GraphQL - Kafka - MCP - Groq

Rapport d’avancement : jalons 1 à 11

Octobre 2026

Réalisé par : Abdelali Saadali

Projet pratique de microservices bancaires

Document de validation technique et de synthèse

1. Présentation du projet

Adria Banking AI est une application bancaire de démonstration organisée en microservices. Le projet vise à isoler la gestion des comptes et des transactions, centraliser l’accès HTTP, assurer la découverte des services et ajouter un assistant conversationnel fondé sur des outils MCP en lecture seule.

Objectifs

• Gérer les comptes bancaires et les transactions dans deux services indépendants.

• Exposer les fonctionnalités par REST, GraphQL et MCP.

• Utiliser Eureka, Spring Cloud Gateway et Config Server.

• Publier les événements de transactions via Kafka et les consommer séparément.

• Interroger les données de démonstration avec un agent Spring AI et Groq.

Démarche

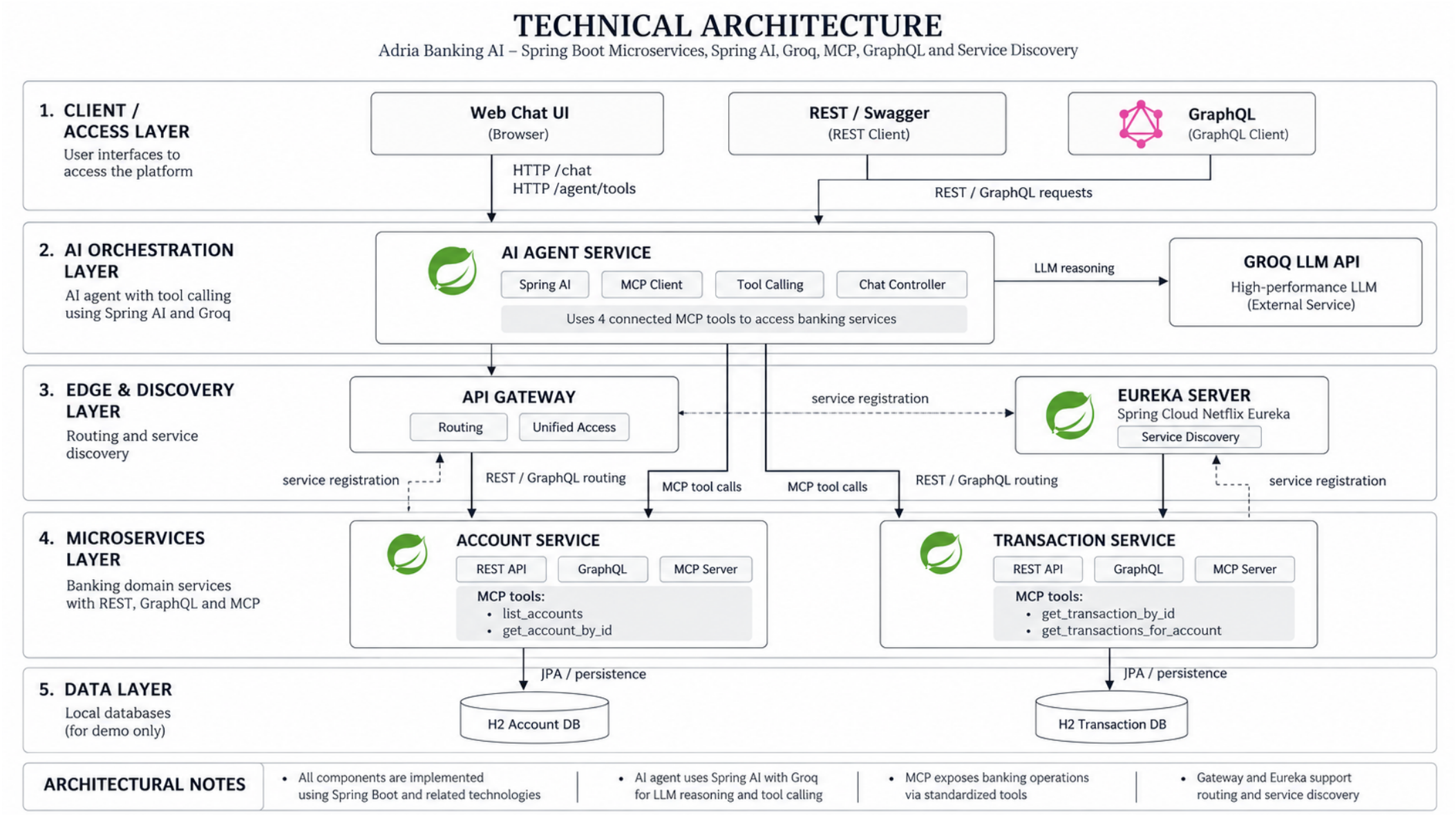
Les modules ont été réalisés par jalons successifs, avec compilation Maven, tests automatisés et essais HTTP ou navigateur. Les captures d’écran annexées correspondent à des étapes observées lors du développement ; les tests ne constituent pas une certification de sécurité ou de conformité bancaire.

Environnement

Java 21, Spring Boot 4.1.2, Spring Cloud 2025.1.4, Spring AI 2.0.2, Maven, bases H2 en mémoire, Docker, Kafka et interface Web HTML sans CSS.

2. Architecture technique

Schéma d’architecture fourni et validé précédemment. Les flux Kafka et Config Server sont détaillés dans les sections suivantes.



3. Services et technologies

Account-Service (8081)

Gestion des comptes : création, consultation, débit, crédit et transfert. Exposition REST, GraphQL et MCP. Persistance JPA dans une base H2 propre au service.

Transaction-Service (8082)

Enregistrement et historique des transactions. Règles de montant, de devise et de disponibilité du compte. Utilisation d’OpenFeign, du LoadBalancer et de Resilience4j pour joindre Account-Service.

Discovery-Service (8761)

Serveur Netflix Eureka : inscription des services et résolution de leurs noms logiques.

Config-Service (8888)

Configuration centralisée ; le plafond des transactions est défini dans le dossier config-repo.

Gateway-Service (8080)

Point d’entrée pour les APIs REST et GraphQL. Routage lb:// vers les instances connues d’Eureka.

Kafka et Transaction-Event-Consumer

Publication des événements TransactionCreated après validation de la transaction locale ; consommation indépendante pour les journaux d’audit.

AI-Agent-Service (8084)

Interface Web en HTML brut, Spring AI, API Groq et quatre outils MCP pour lire les comptes et les transactions.

4. Interfaces et scénario de validation

Entrées HTTP et intégrations

REST

Création et lecture de comptes, enregistrement des transactions et historique des opérations.

GraphQL

Interrogation structurée des ressources bancaires avec GraphiQL.

MCP

Outils read-only : list_accounts, get_account_by_id, get_transaction_by_id, get_transactions_for_account.

Kafka

Sujet bank.transactions.created ; message envoyé par le producteur, puis reçu par un service consommateur distinct.

Groq

Modèle qwen/qwen3.8-27b appelé à travers une interface compatible OpenAI. La clé d’accès reste à l’extérieur du dépôt Git.

Scénarios réalisés

Les essais ont validé notamment le routage par Gateway, la découverte Eureka, le débit d’un compte, la réception d’un événement Kafka et le fonctionnement de l’agent MCP. L’agent a pu lire un compte de démonstration et répondre à une question sur ses transactions.

5. Résultats, limites et perspectives

Résultats de validation

• Construction des modules par Maven ; vérifications automatisées effectuées au fil des jalons.

• 16 tests réussis sur Transaction-Service après l’ajout du producteur Kafka.

• 2 tests du consommateur de transactions exécutés avec succès.

• 8 tests réussis sur AI-Agent-Service après correction de la gestion des appels d’outils invalides.

• Essais réels : lecture des outils MCP, question au modèle Groq, et publication / consommation Kafka.

Limites identifiées

• H2 fonctionne en mémoire et les données de démonstration disparaissent au redémarrage.

• L’appel distant et l’écriture dans deux bases ne forment pas une transaction distribuée atomique.

• Un producteur Kafka après commit ne garantit pas à lui seul la livraison en cas de panne ; un outbox est une amélioration prévue.

• Les endpoints locaux et le chatbot ne sont pas encore protégés par authentification et autorisation complètes.

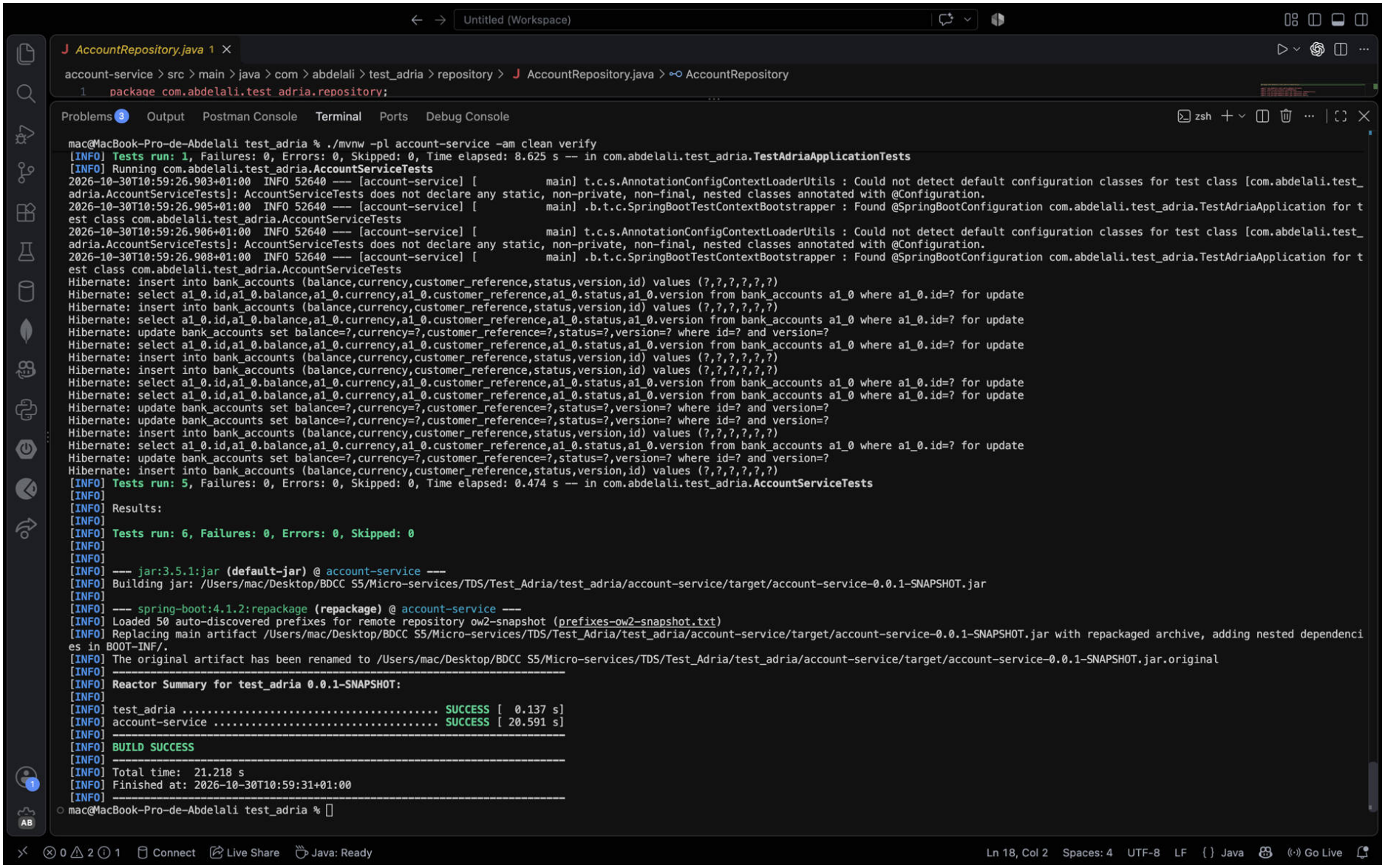
• Les fonctionnalités de sécurité Keycloak, le frontend Angular et la détection de fraude restent des travaux futurs.

Conclusion

Les jalons présentés constituent un socle microservices fonctionnel pour un projet pédagogique. Les composants métier, la découverte, le routage, l’échange événementiel et l’assistant IA sont documentés ci-après par les captures originales.

Annexe - capture S01

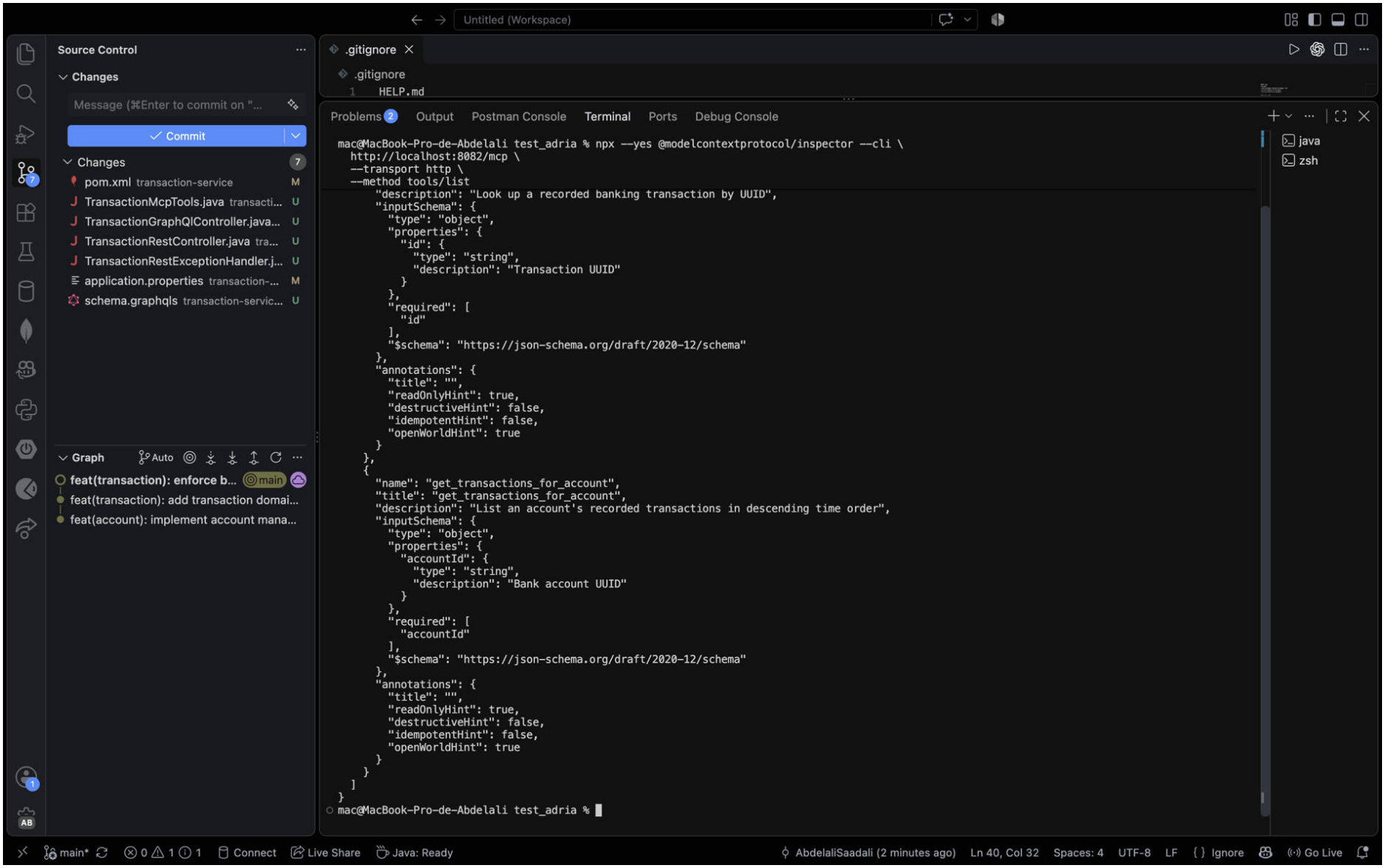
Sortie Maven / console de validation



S01 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S02

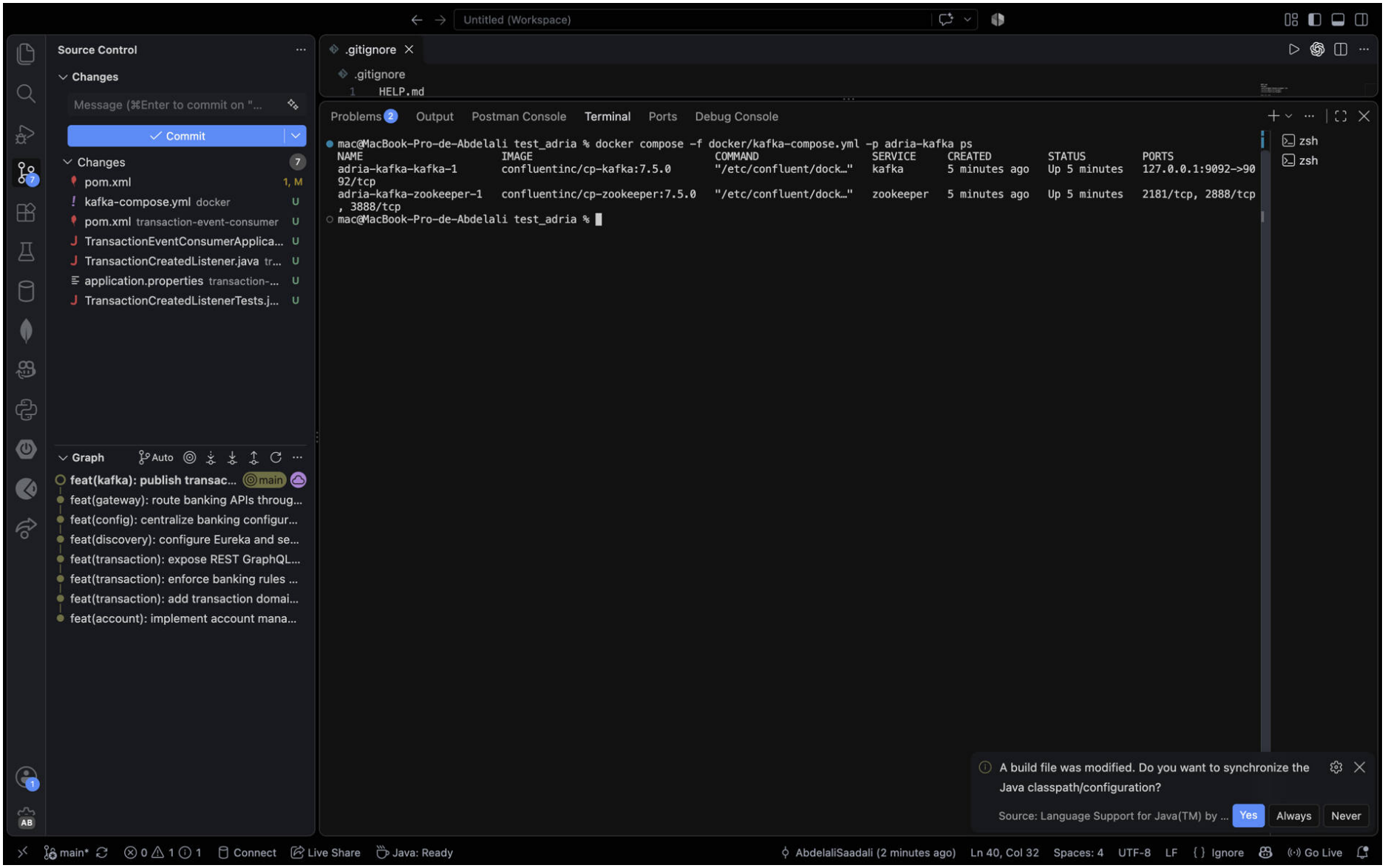
Fichier de configuration et structure du projet



S02 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S03

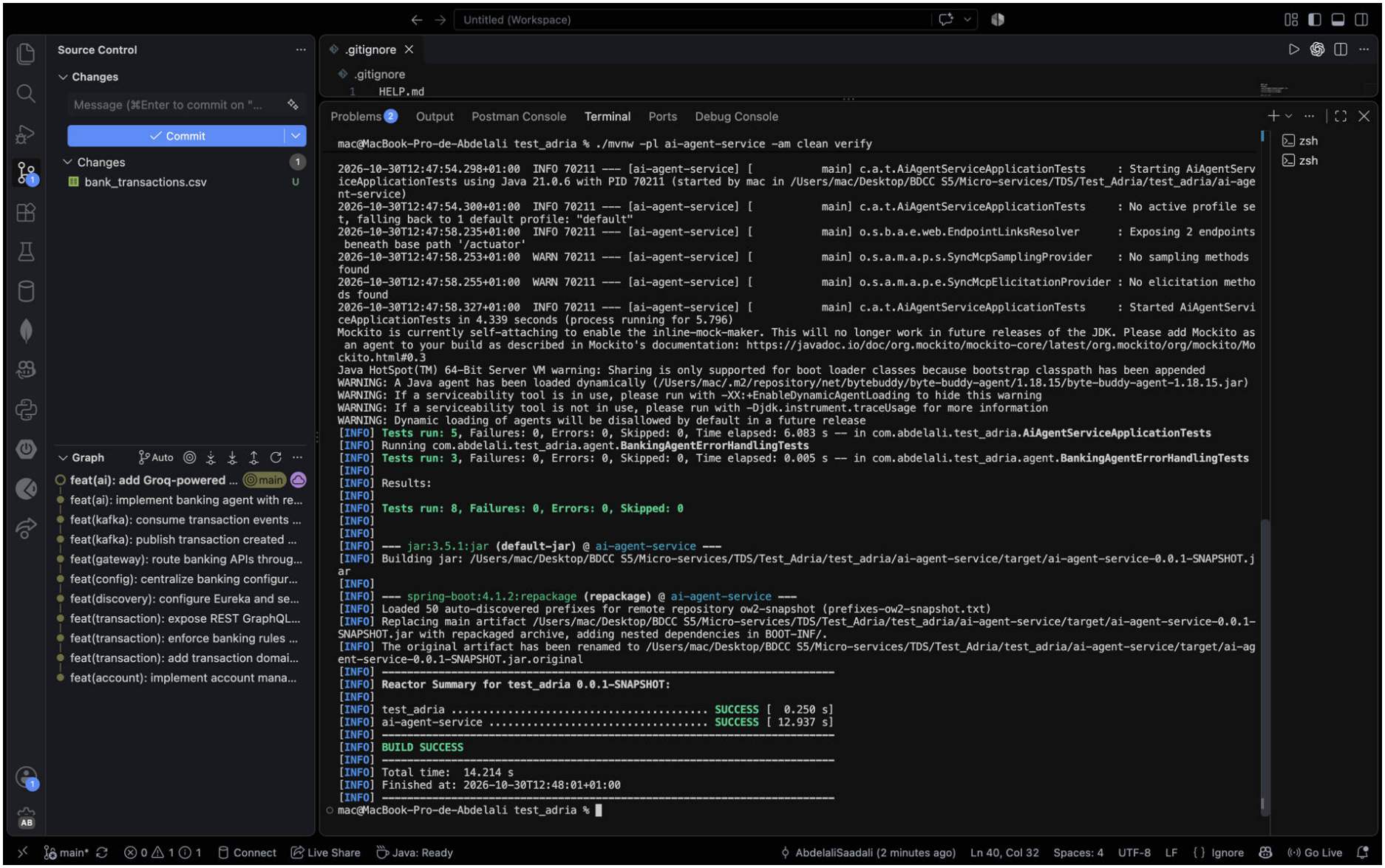
Inspection d’une configuration ou d’une réponse technique



S03 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S04

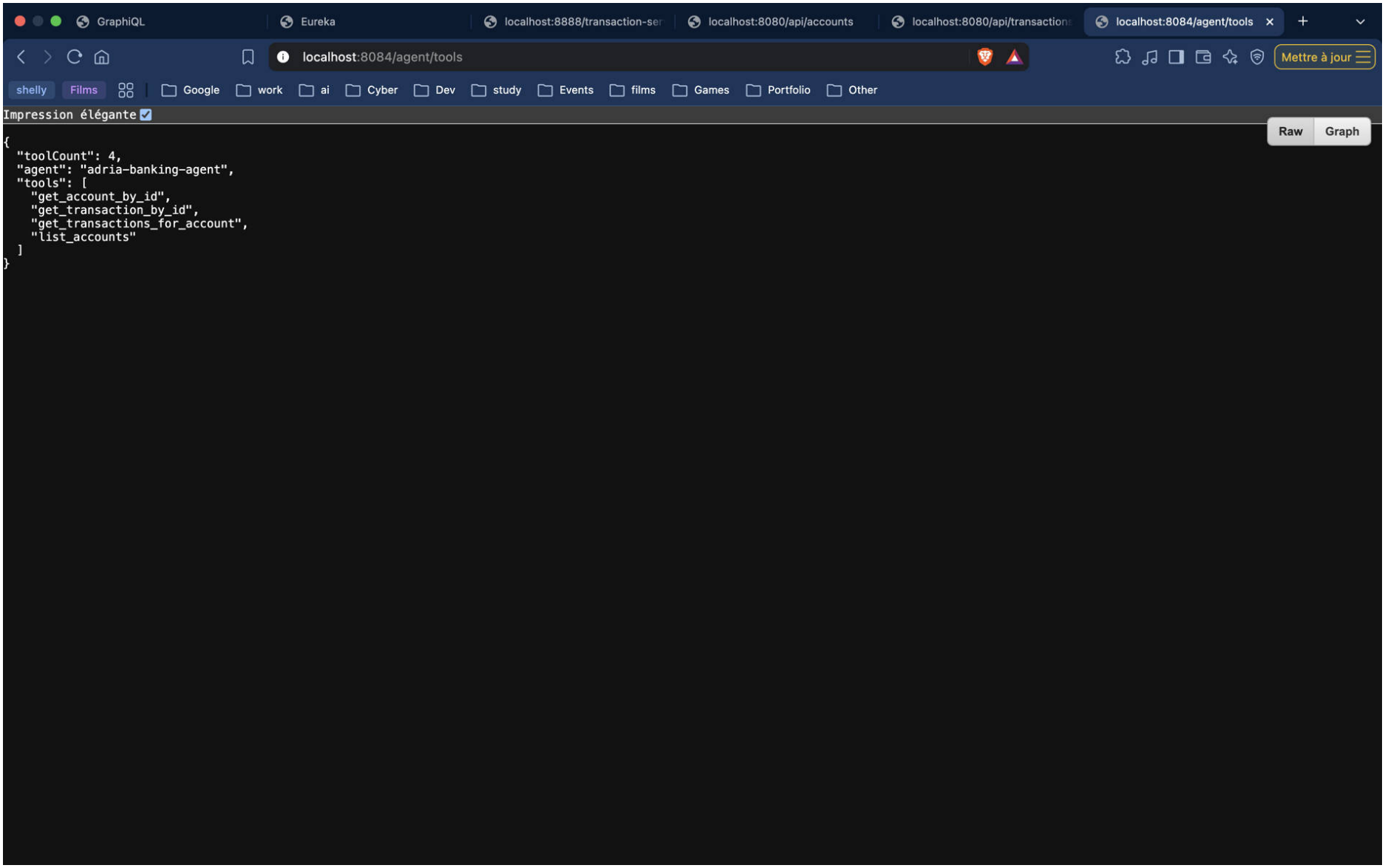
Exécution de tests dans l’environnement de développement



S04 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S05

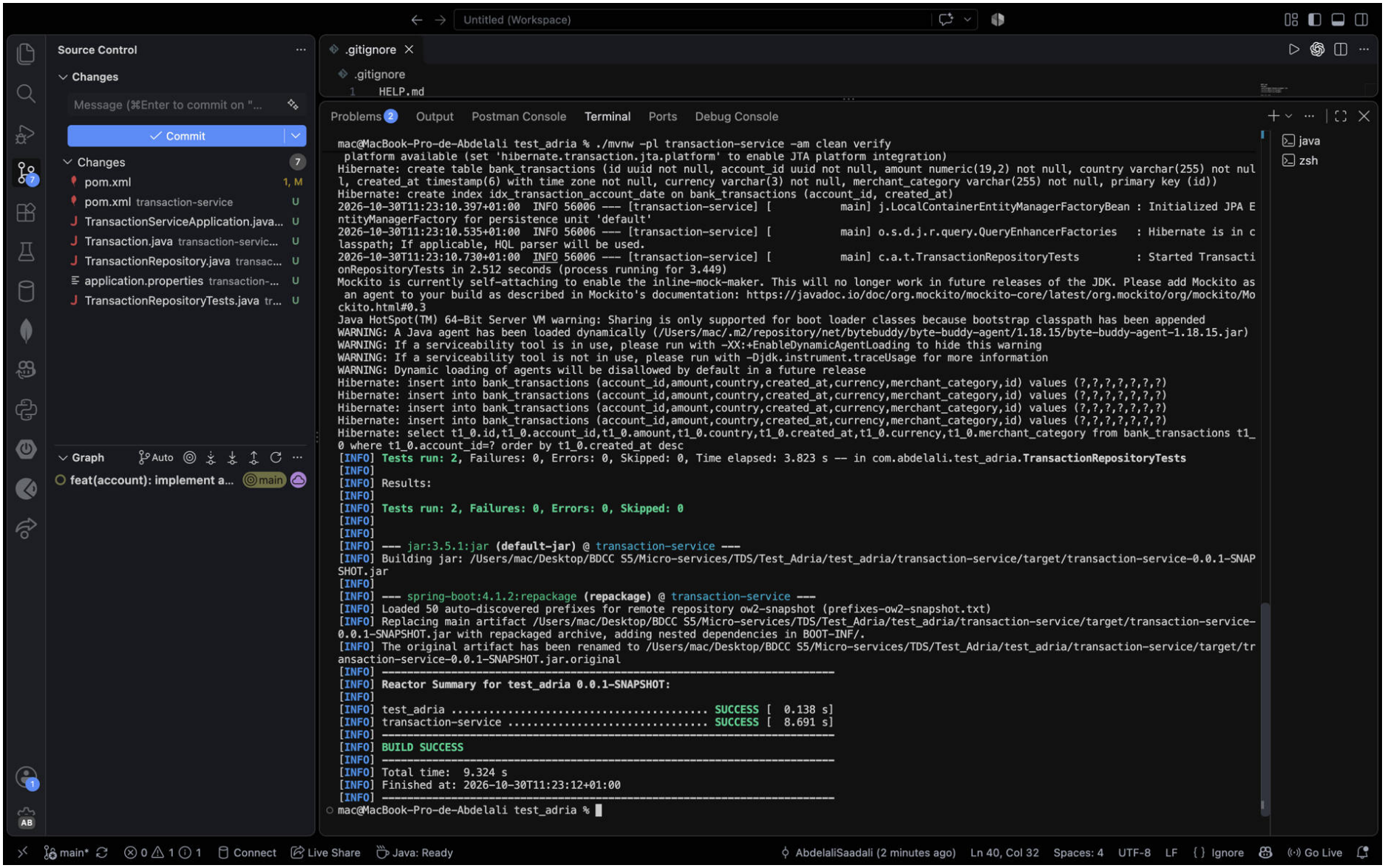
Réponse HTTP observée dans le navigateur



S05 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S06

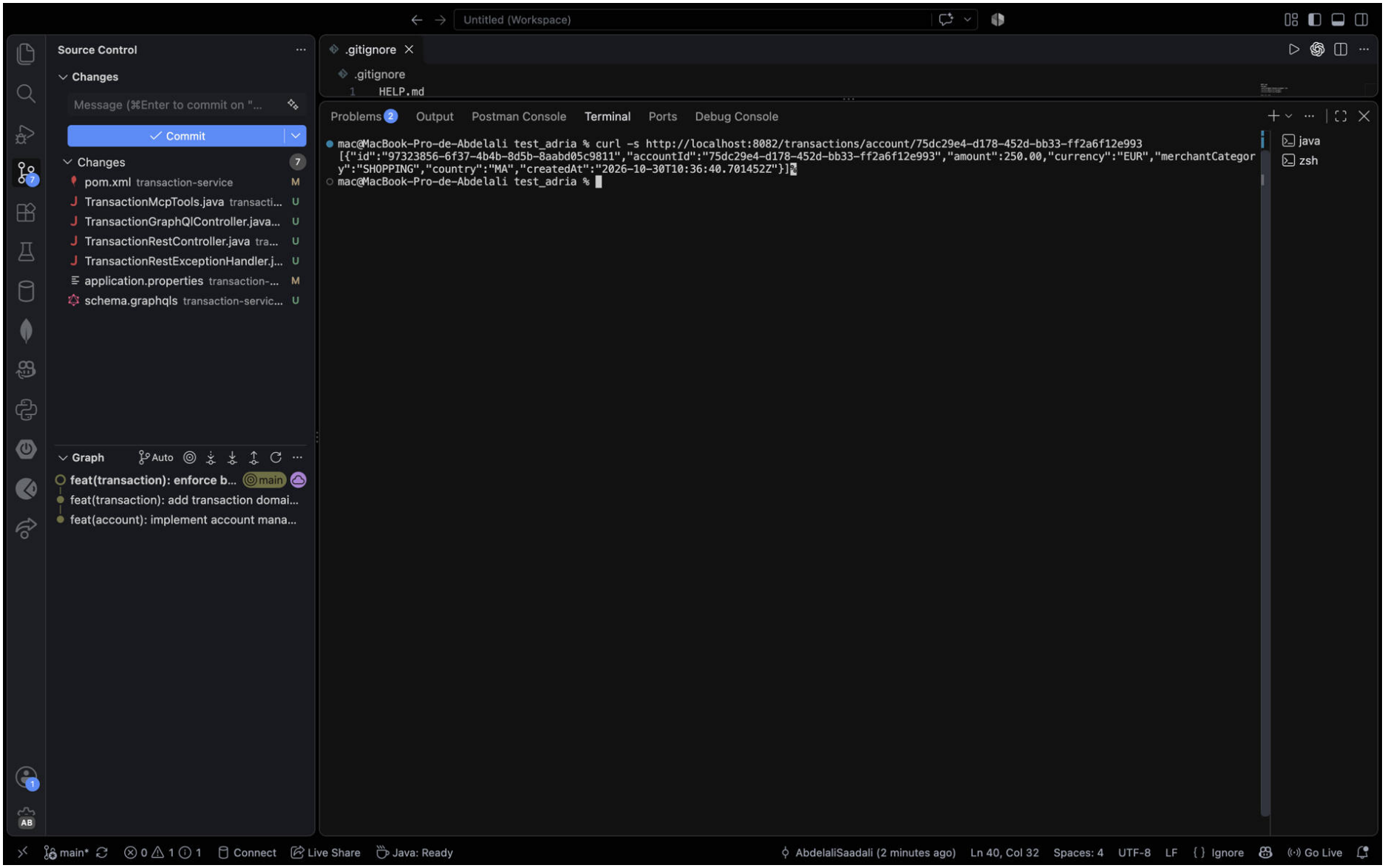
Résultats de tests Maven



S06 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S07

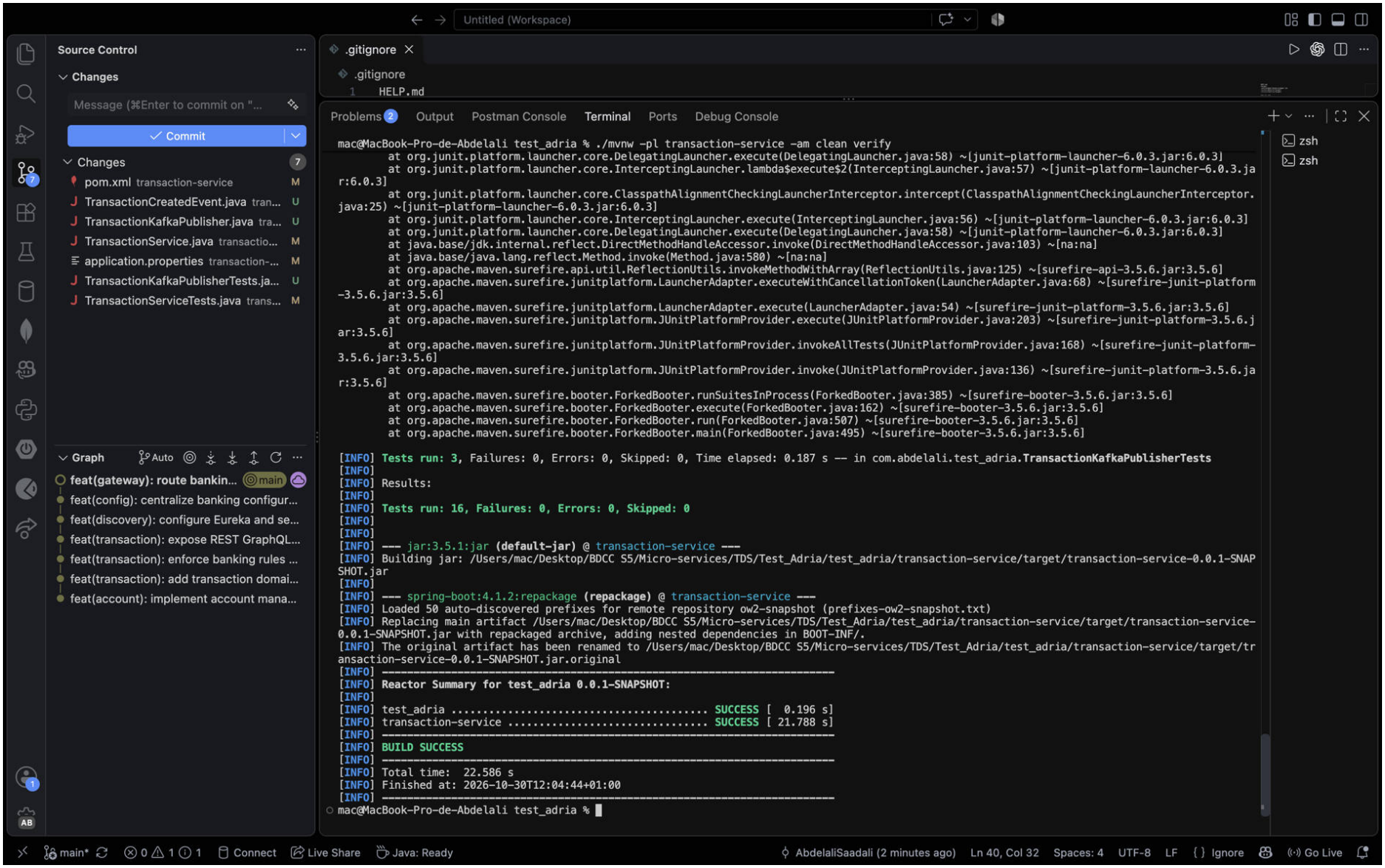
Consultation de code et de configuration



S07 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S08

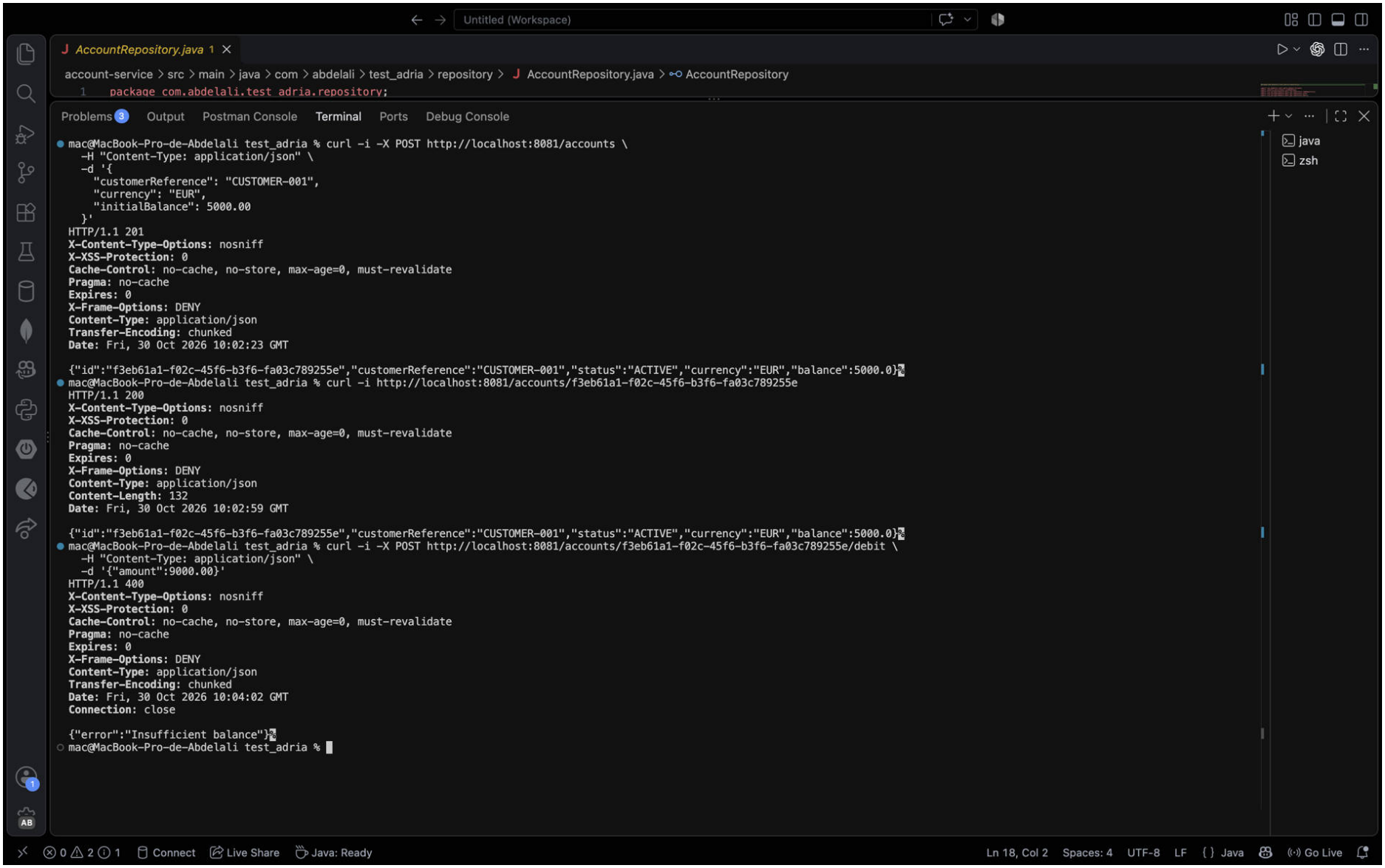
Exécution de tests et journal applicatif



S08 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S09

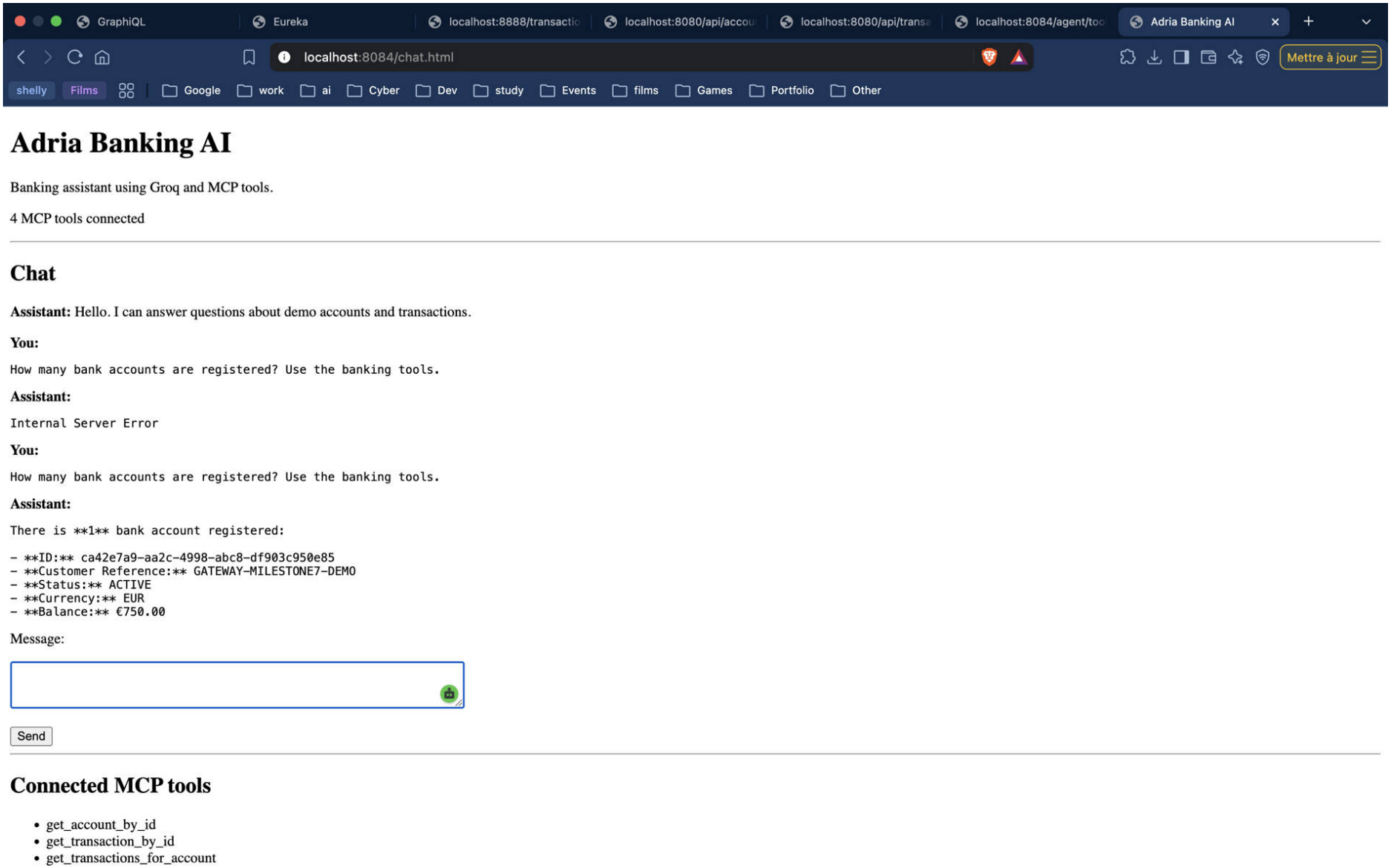
Fichier de configuration du service



S09 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S10

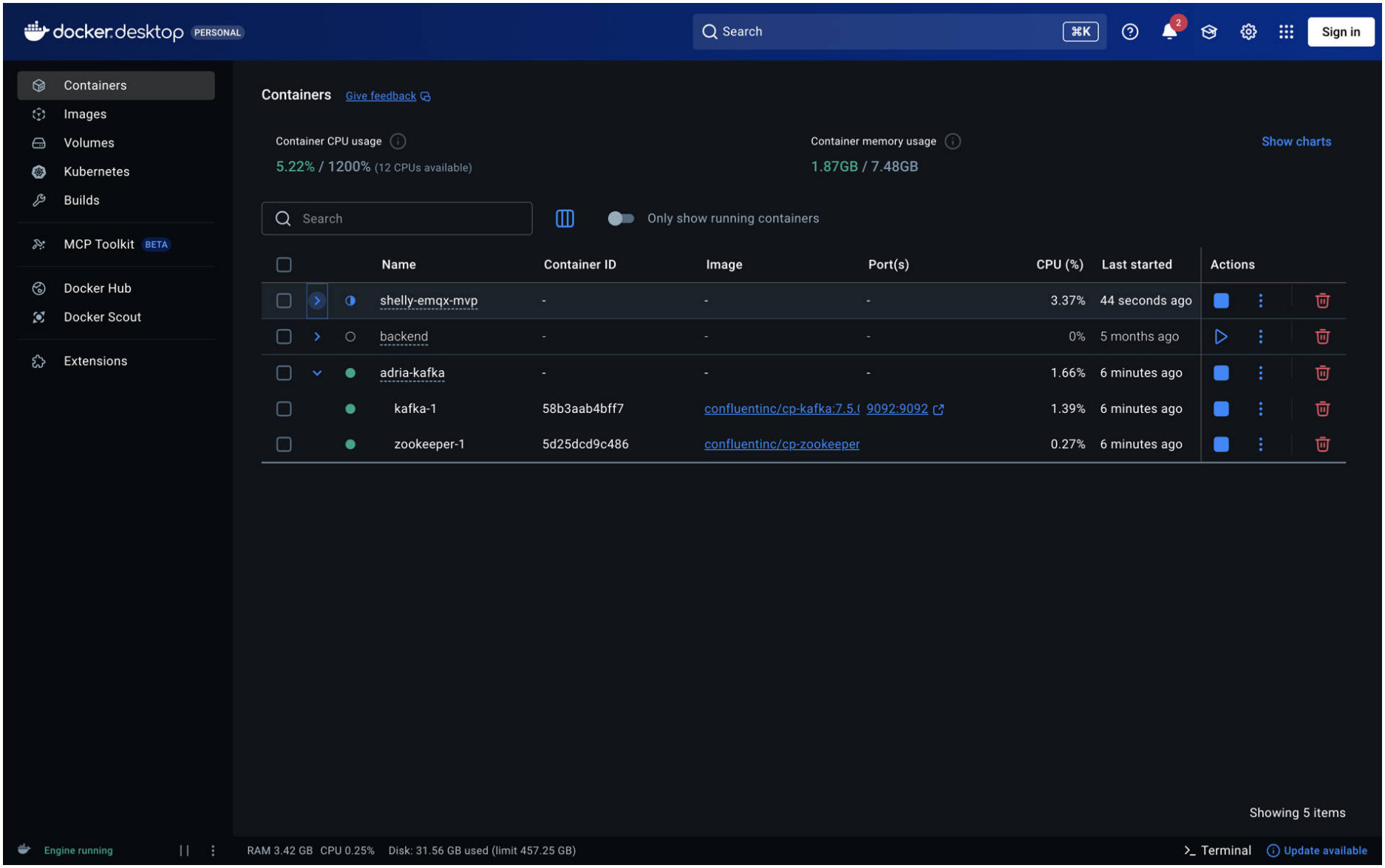
Interface de chat, quatre outils MCP et réponse du modèle



S10 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S11

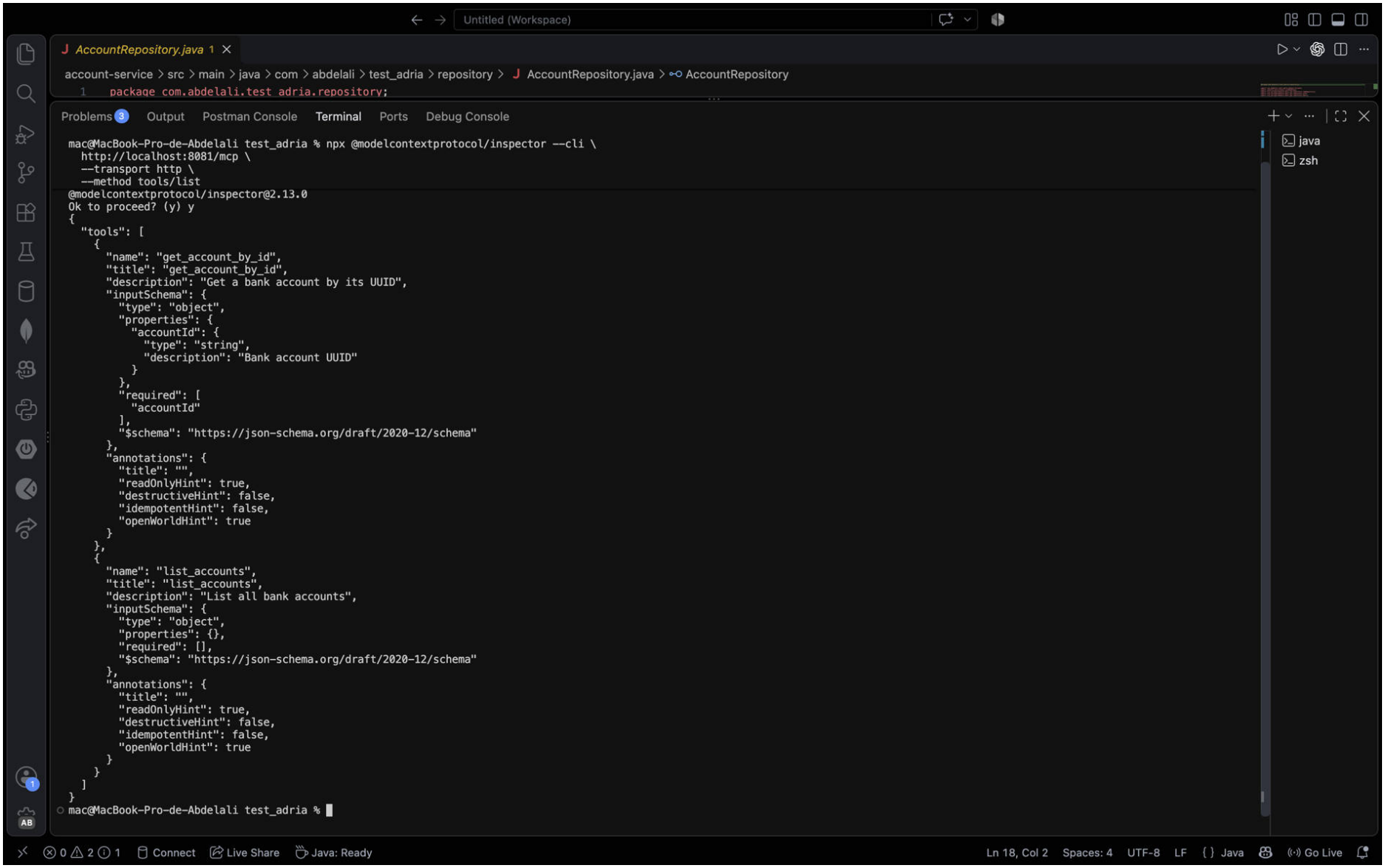
Conteneurs de l’environnement Docker



S11 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S12

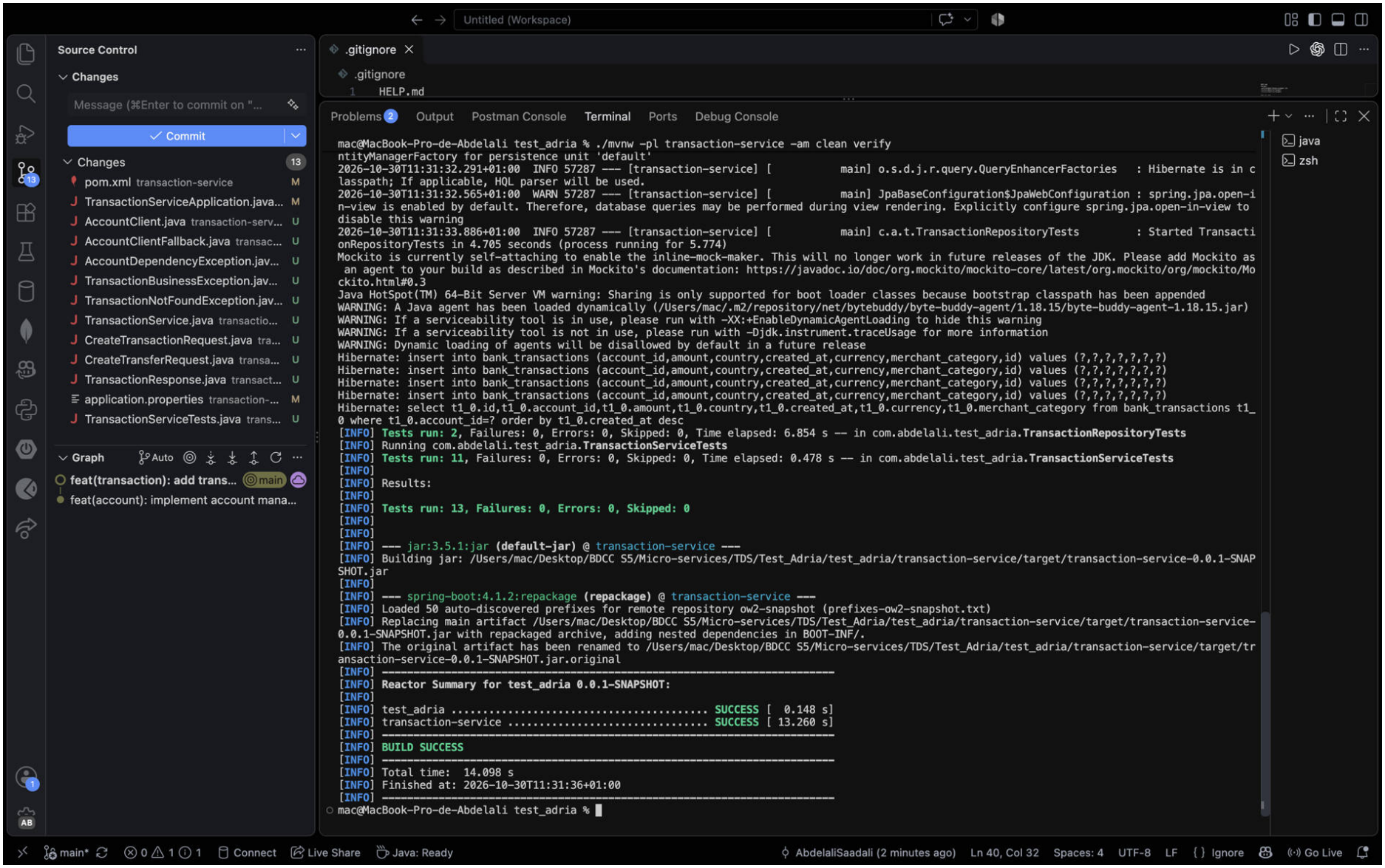
Consultation technique dans l’éditeur



S12 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S13

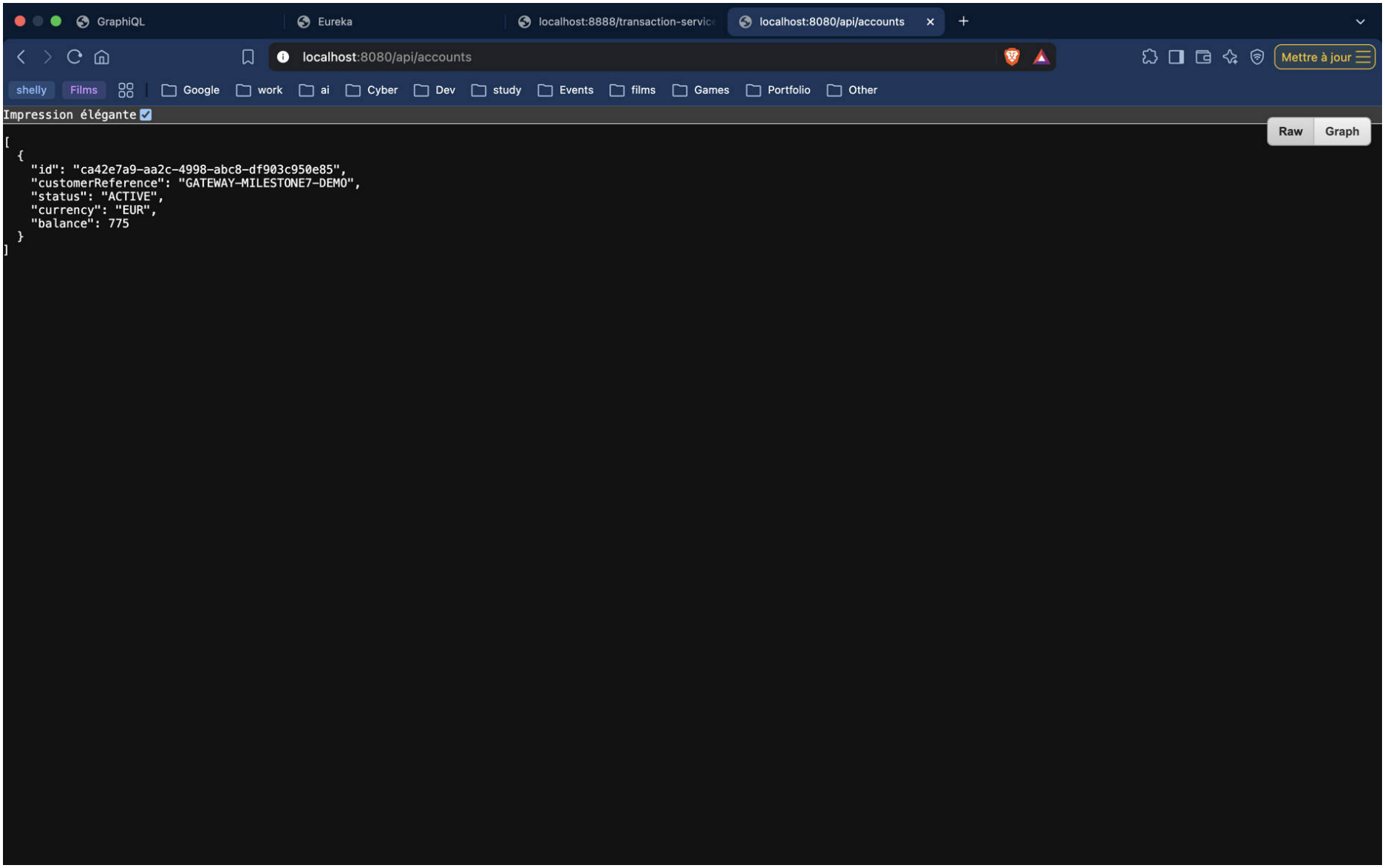
Tests d’intégration en console



S13 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S14

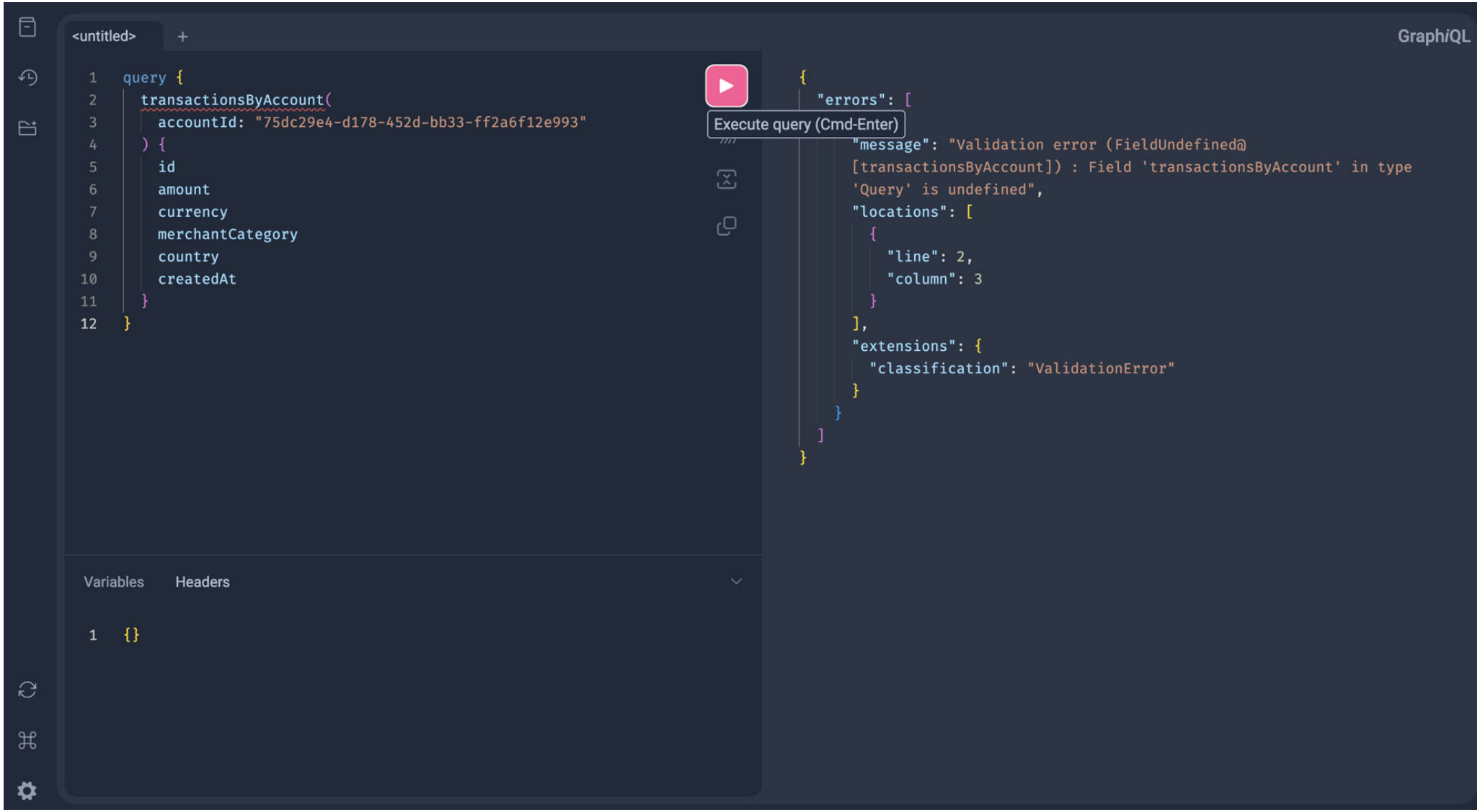
Consultation d’une réponse API dans le navigateur



S14 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S15

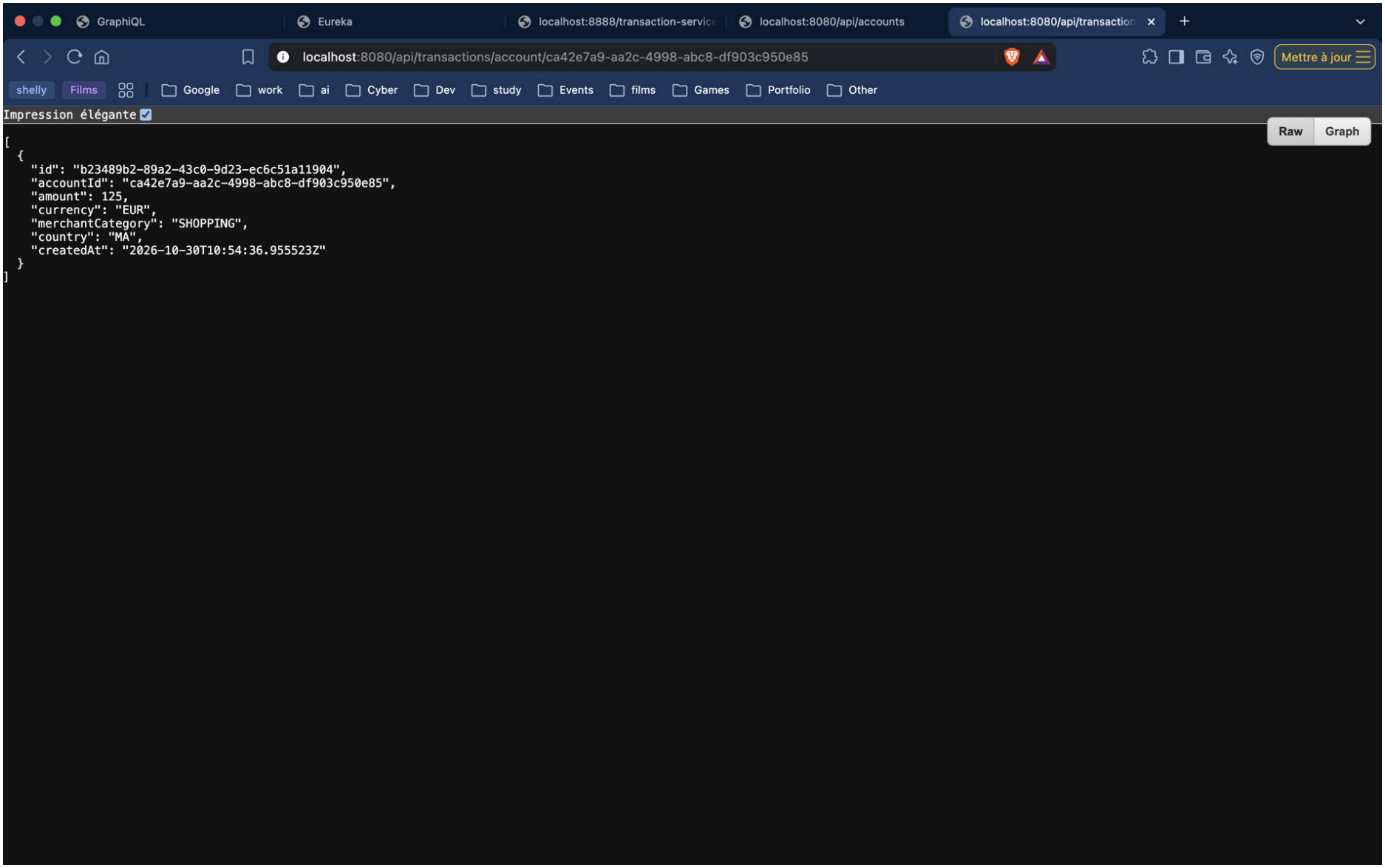
Essai GraphQL dans GraphiQL



S15 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S16

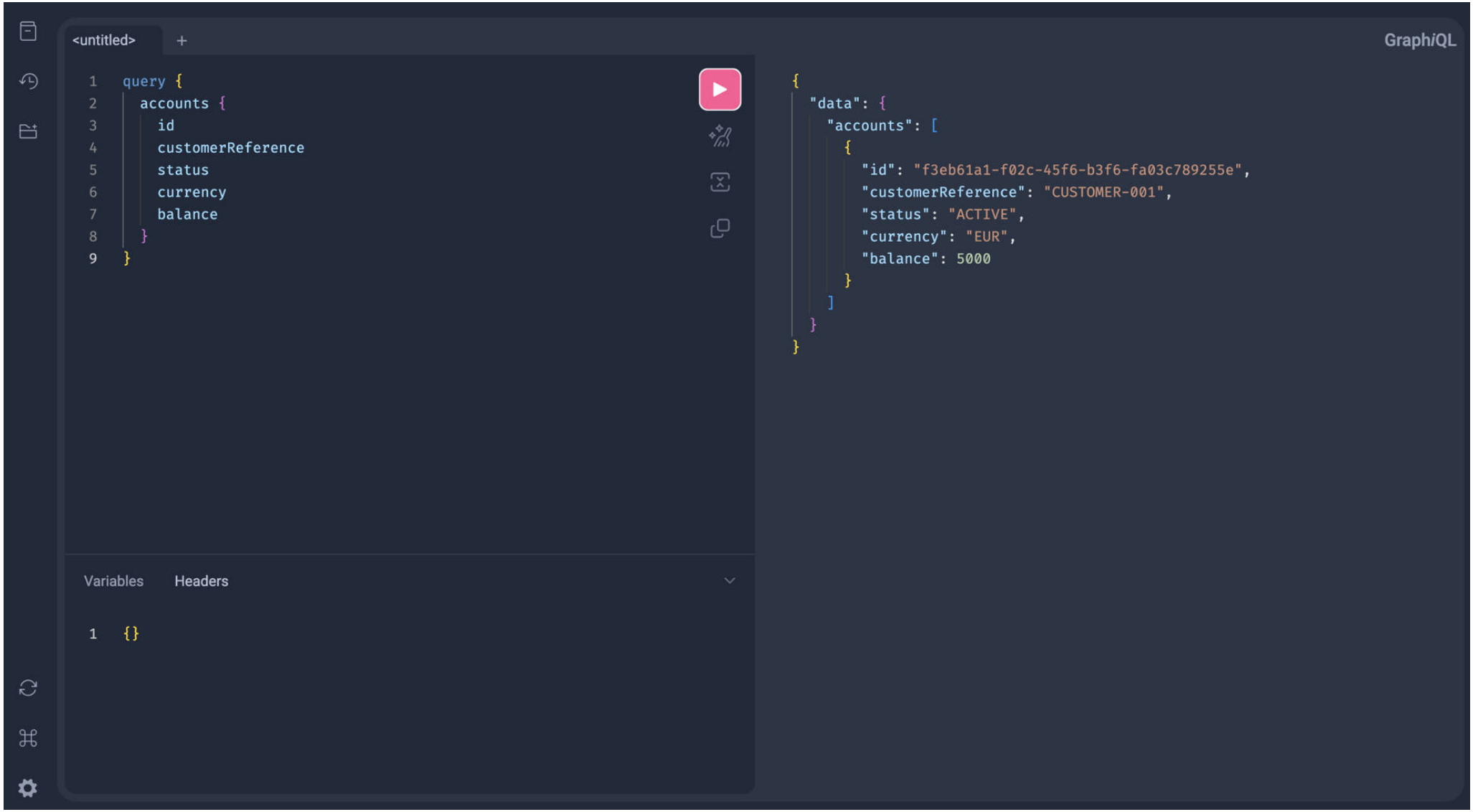
Consultation d’un endpoint HTTP



S16 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S17

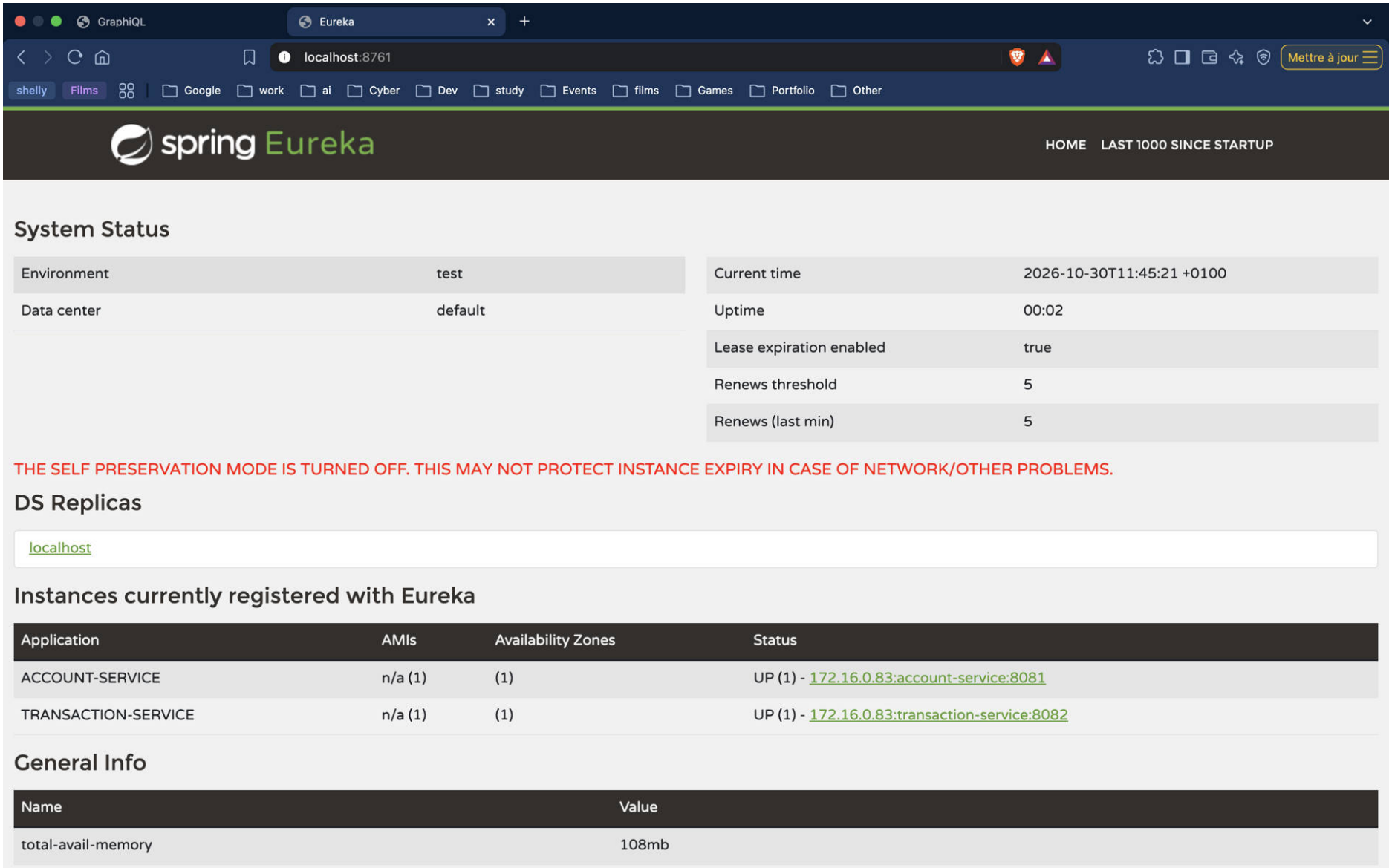
Requête GraphQL et résultat associé



S17 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S18

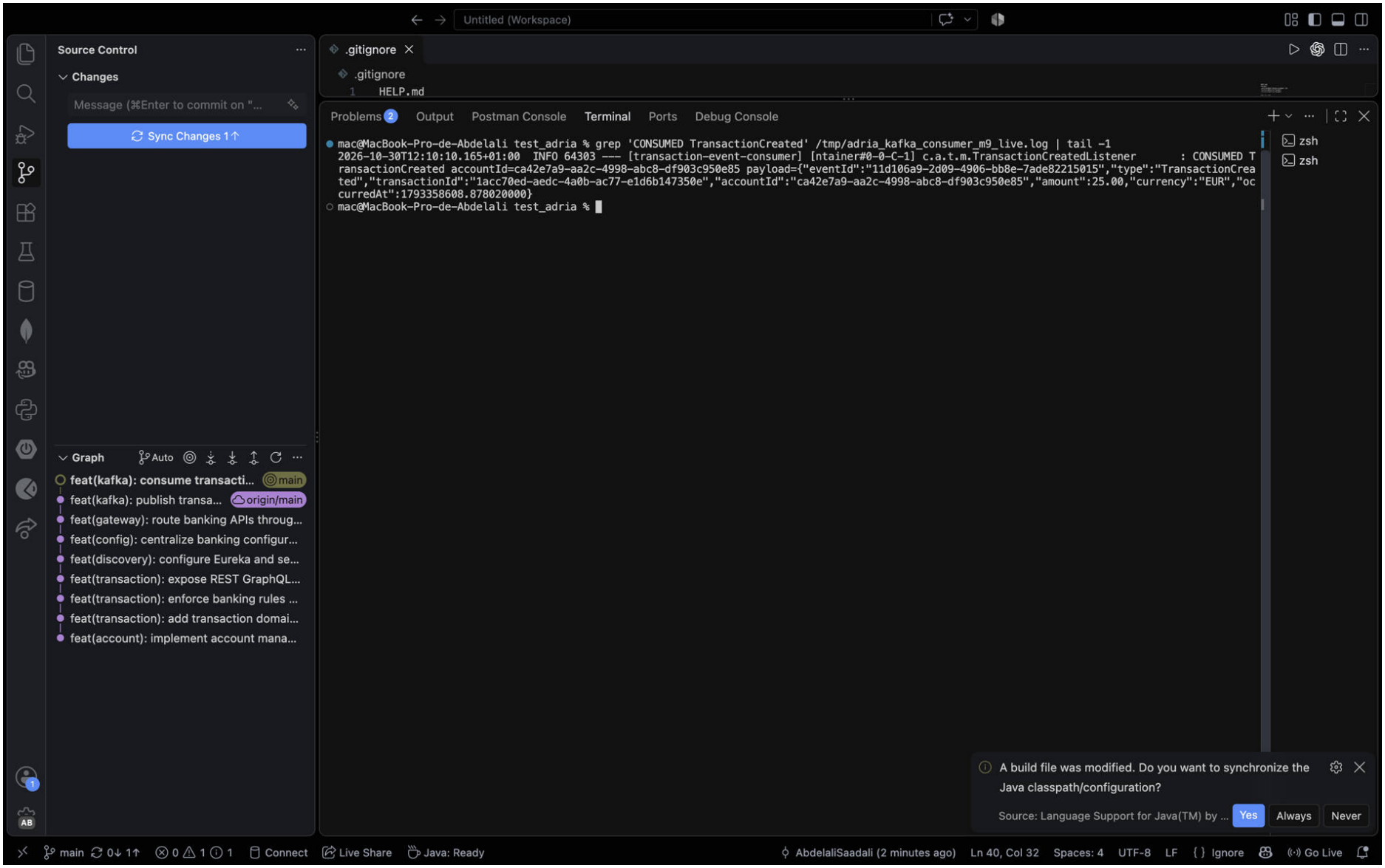
Tableau de bord du serveur Eureka



S18 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S19

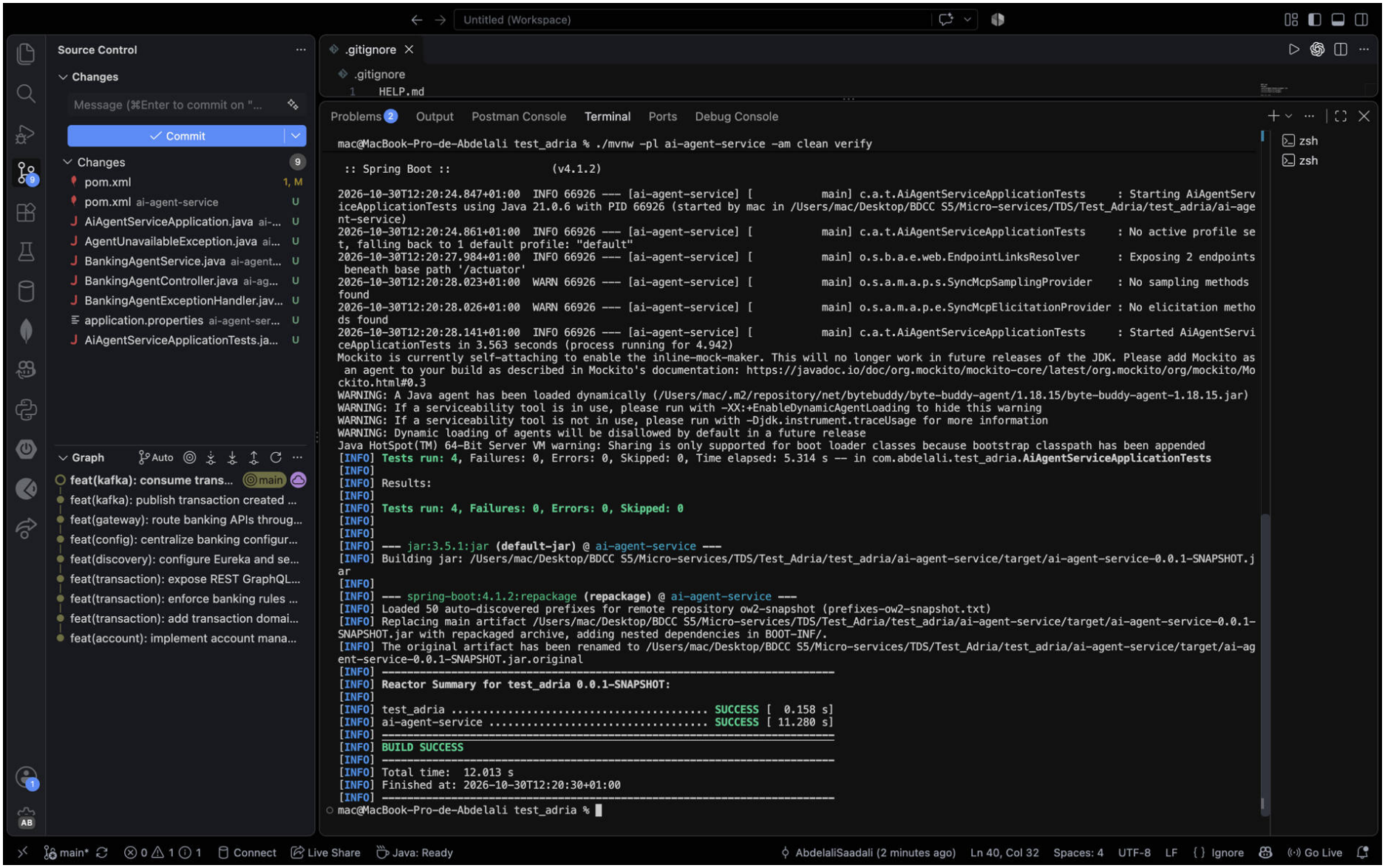
Journal d’exécution et de tests



S19 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S20

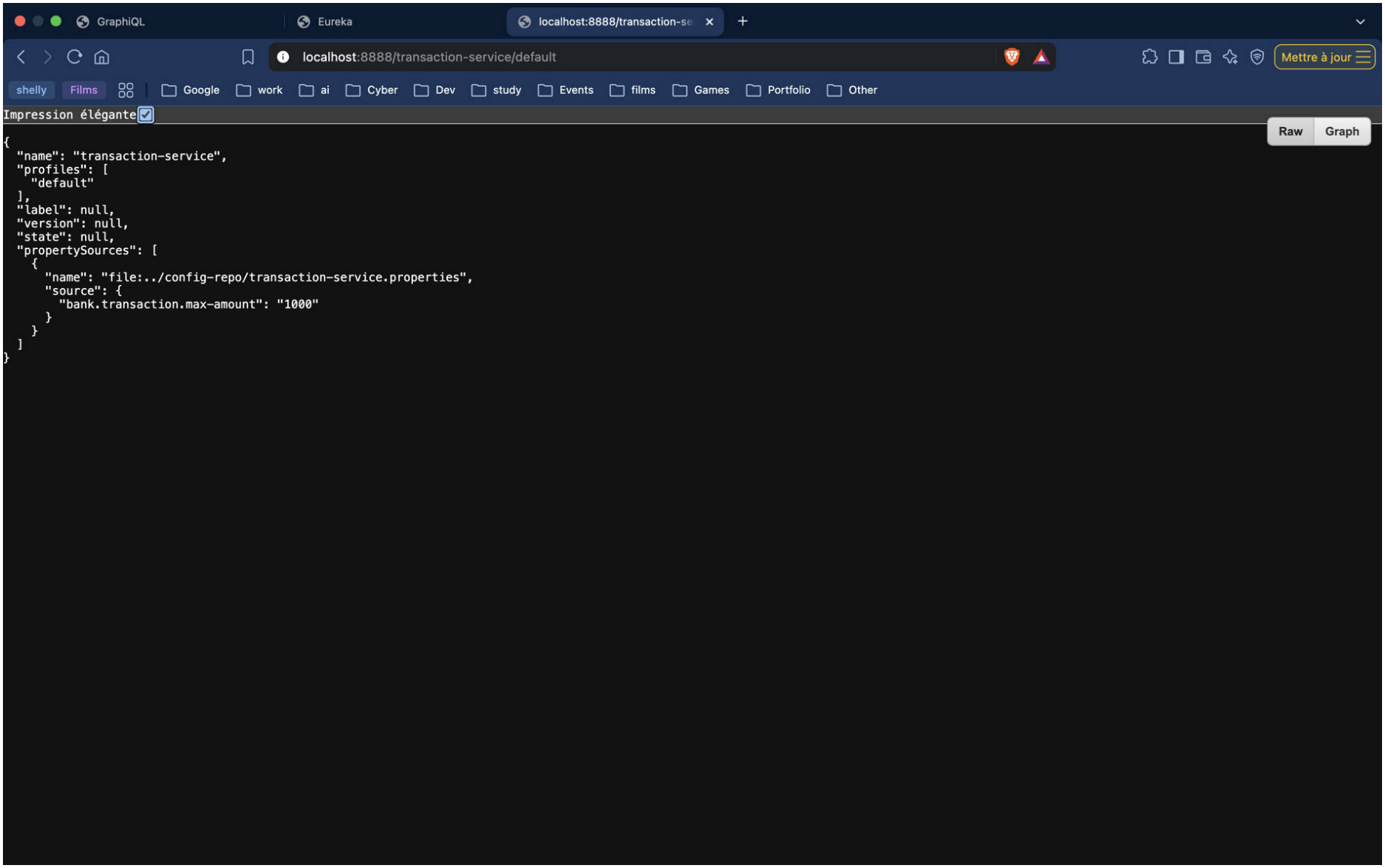
Compilation et résultats de tests



S20 - Capture fournie dans l’archive screenshots(4).zip

Annexe - capture S21

Résultat d’un appel HTTP dans le navigateur



S21 - Capture fournie dans l’archive screenshots(4).zip